JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicReference<V>

java.lang.Object
  java.util.concurrent.atomic.AtomicReference<V>

Type Parameters:

V - The type of object referred to by this reference

All Implemented Interfaces:

Serializable

```
public class AtomicReference<V>
extends Object
implements Serializable
```

An object reference that may be updated atomically. See the VarHandle specification for descriptions of the properties of atomic accesses.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
AtomicReference()	Creates a new AtomicReference with null initial value.
AtomicReference(V initialValue)	Creates a new AtomicReference with the given initial value.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
final V	accumulateAndGet(V x, BinaryOperator<V> accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying

		the given function to the current and given values, returning the updated value.
final V	**compareAndExchange**(V expectedValue, V newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchange(Object...).
final V	**compareAndExchangeAcquir**(V expectedValue, V newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final V	**compareAndExchangeReleas**(V expectedValue, V newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeRelease(Obje
final boolean	**compareAndSet**(V expectedValue, V newValue)	Atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).

final V	**get**()	Returns the current value, with memory effects as specified by VarHandle.getVolatile(Object...).
final V	**getAcquire**()	Returns the current value, with memory effects as specified by VarHandle.getAcquire(Object...).
final V	**getAndAccumulate**(V x, BinaryOperator<V> accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function to the current and given values, returning the previous value.
final V	**getAndSet**(V newValue)	Atomically sets the value to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).
final V	**getAndUpdate**(UnaryOperator<V> update	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function, returning the previous value.
final V	**getOpaque**()	Returns the current value, with memory effects as specified by VarHandle.getOpaque(Object...).
final V	**getPlain**()	Returns the current value, with memory semantics of reading as

		if the variable was declared non-volatile.
final void	**lazySet**(V newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
final void	**set**(V newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque**(V newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setOpaque(Object...).
final void	**setPlain**(V newValue)	Sets the value to newValue, with memory semantics of setting as if the variable was declared non-volatile and non-final.
final void	**setRelease**(V newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
String	**toString**()	Returns the String representation of the current value.
final V	**updateAndGet**(UnaryOperator<V> update	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function, returning the updated value.

final boolean	**weakCompareAndSet**(V expectedValue, V newValue)	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as compareAndExchange(V, V) and compareAndSet(V, V)).
final boolean	**weakCompareAndSetAcquire**(V expectedValue, V newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain**(V expectedValue, V newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease**(V expectedValue, V newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatile**(V expectedValue, V newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicReference

```
public AtomicReference(V initialValue)
```

Creates a new AtomicReference with the given initial value.

Parameters:

`initialValue` - the initial value

AtomicReference

```
public AtomicReference()
```

Creates a new AtomicReference with null initial value.

Method Details

get

```
public final V get()
```

Returns the current value, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Returns:

the current value

set

```
public final void set(V newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.

Parameters:

`newValue` - the new value

lazySet

```
public final void lazySet(V newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

1.6

compareAndSet

```
public final boolean compareAndSet(V expectedValue,
                                   V newValue)
```

Atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(V expectedValue,
                                       V newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(V, V)` and `compareAndSet(V, V)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(V, V)` be used instead.

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(V, V)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(V expectedValue,
                                            V newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndSet

```
public final V getAndSet(V newValue)
```

Atomically sets the value to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`newValue` - the new value

Returns:

the previous value

getAndUpdate

```
public final V getAndUpdate(UnaryOperator<V> updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final V updateAndGet(UnaryOperator<V> updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final V getAndAccumulate
(V x,
 BinaryOperator<V> accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final V accumulateAndGet
(V x,
 BinaryOperator<V> accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

getPlain

```
public final V getPlain()
```

Returns the current value, with memory semantics of reading as if the variable was declared non-`volatile`.

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(V newValue)
```

Sets the value to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`newValue` - the new value

Since:

9

getOpaque

```
public final V getOpaque()
```

Returns the current value, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(V newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

getAcquire

```
public final V getAcquire()
```

Returns the current value, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(V newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

compareAndExchange

```
public final V compareAndExchange(V expectedValue,
                                  V newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final V compareAndExchangeAcquire(V expectedValue,
                                         V newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final V compareAndExchangeRelease(V expectedValue,
                                         V newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(V expectedValue,
                                               V newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(V expectedValue,
                                              V newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(V expectedValue,
                                              V newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.